

Reviewer Pack — Minimal Order of Formal Contexts and Communication Complexit...

Entry cd1d49137012 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 18:33:18 UTC

Minimal Order of Formal Contexts and Communication Complexity Rank Variance Ratio

Entry ID: cd1d49137012 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 18:33:18 UTC

1. Conjecture

Field A (mathematical branch): Lattice Theory (Formal Contexts) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every n -ary Boolean function f , the ratio of the minimal order of a formal context that represents f to the communication complexity rank of the matrix representation of f is upper-bounded by $O(\log n)$. Equivalently: $|O(F_n)| / |R_f| \leq O(\log n)$, where $O(F_n)$ is the minimal order of a formal context representing an n -ary function f , and R_f is its matrix representation.

Rationale (proposer's reasoning):

Formal contexts provide a relational structure that can be used to represent Boolean functions. The communication complexity of a function is related to the complexity of computing the function in the presence of noise or when parties have limited information about each other. This conjecture suggests that there is an inherent relationship between these two areas, which could lead to new insights into the nature of computation.

Taxonomy category: INFORMATION_THEORY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 62ddd10fef066f97

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The ratio of the minimal order of a formal context to the communication complexity rank is within $O(\log n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - formal contexts AND communication complexity rank variance - lattice theory AND matrix representation communication complexity - minimal order formal context ratio communication complexity rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_truth_table(n):
    return [[random.randint(0, 1) for _ in range(2**n)] for _ in range(2**n)]

def calculate_minimal_order(truth_table):
    n = len(truth_table)
    rows = truth_table[:]
    cols = [sum(row[i] for row in rows) for i in range(n)]

    minimal_order = 0
    while rows:
        min_row_index = min(range(len(rows)), key=lambda i: sum(rows[i]))
        min_col_index = min(range(n), key=lambda j: sum(row[j] for row in rows))

        if rows[min_row_index][min_col_index] == 1:
            minimal_order += 1
            rows.pop(min_row_index)
            for row in rows:
                row[min_col_index] = 0
        else:
            break

    return minimal_order

def calculate_rank(matrix):
    n = len(matrix)
    rank = 0
    matrix_copy = [row[:] for row in matrix]

    for i in range(n):
```

```

    if matrix_copy[i][i] == 0:
        found_pivot = False
        for j in range(i+1, n):
            if matrix_copy[j][i] != 0:
                matrix_copy[i], matrix_copy[j] = matrix_copy[j], matrix_copy[i]
                found_pivot = True
                break
        if not found_pivot:
            continue

    rank += 1
    for j in range(n):
        if j == i:
            continue
        factor = Fraction(matrix_copy[j][i], matrix_copy[i][i])
        for k in range(n):
            matrix_copy[j][k] -= factor * matrix_copy[i][k]

    return rank

def calculate_ratio(truth_table):
    n = len(truth_table)
    minimal_order = calculate_minimal_order(truth_table)
    rank = calculate_rank(truth_table)
    if rank == 0:
        return None
    return Fraction(minimal_order, rank)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        truth_table = generate_truth_table(n)
        ratio = calculate_ratio(truth_table)

        if ratio is None or ratio == Fraction(0, 0):
            return {
                "metric_name": "Ratio",
                "metric_value": None,
                "instances_tested": 1,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": "Matrix is singular"
            }

        results.append(ratio)

    mean_ratio = sum(results) / len(results)
    conjecture_holds = all(math.log(n) >= ratio for n, ratio in zip(n_values, results))

    return {
        "metric_name": "Ratio",
        "metric_value": float(mean_ratio),

```

```

        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction:.2f}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='Ratio exceeds log(n)' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2290349b.py", line 116, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2290349b.py", line 85, in run_trial
    if ratio is None or ratio == Fraction(0, 0):
                           ~~~~~
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that no ratios were calculated to evaluate against the conjecture’s conditions. | next: Re-run the test with proper error handling and ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15385
2	preregistration	ollama_remote	glm4:latest	0	0	9339
3	novelty	ollama_remote	glm4:latest	0	0	17087
4	novelty	ollama_remote	glm4:latest	0	0	8968
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33945
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12582
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12978
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11655
9	judge	ollama_remote	glm4:latest	0	0	16337

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 138276 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/cd1d49137012.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/cd1d49137012.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/cd1d49137012.tar.gz (if generated)